REST ASSURED

# Class 2: Complete Beginner Notes

## Response Assertions • Hamcrest • POST Sign-in • JSON Schema • JsonPath

A step-by-step practical guide with line-by-line code explanations, real-world QA scenarios, common mistakes, hands-on exercises, and worked solutions.

**Learning approach** First understand why a test is needed, then read the code, predict the result, run it, and finally change one condition to see the test fail.

### By the end, you should be able to

- Explain request, response, status code, headers, response body, assertion, and schema in simple terms.
- Use Hamcrest matchers to verify values, ranges, strings, collections, keys, and data types.
- Send a POST sign-in request using a JSON payload file and validate the returned response.
- Validate response structure with a JSON Schema file.
- Extract root-level, nested, and array values with REST Assured JsonPath.
- Write cleaner TestNG tests and debug failed assertions like an API automation engineer.

Based on the uploaded “Rest Assured class 2(2).txt” notes. Additional examples are illustrative; use the response contract of your own API when deciding expected status codes, headers, and fields.

# How to use these notes

| Stage | What to do | Done when… |
|---|---|---|
| 1. Understand | Read the simple explanation before the code. | You can explain the purpose without memorising lines. |
| 2. Type | Type the code in your own IDE rather than copy-pasting everything. | The code compiles in your project. |
| 3. Run | Execute the test and inspect the TestNG result. | You understand why it passed or failed. |
| 4. Break it | Change an expected value, header, field name, or threshold. | You can predict and explain the failure. |
| 5. Practise | Complete the exercises at the end before opening the answers. | You can build similar tests independently. |

## What is covered from the class file?

The uploaded material focuses on verifying status code, status line, response time, headers, response payload, and JSON Schema; it introduces Hamcrest matchers, a POST sign-in example, schema validation, and JsonPath getters.

## A simple mental model

```
Test code  -> sends HTTP request -> API processes it -> API returns response -> assertions compare actual vs expected
```

| Term | Beginner-friendly meaning | Example |
|---|---|---|
| Request | What the test sends to the API. | “Please sign in with this email and password.” |
| Response | What the API sends back. | Status 201 and JSON with user ID and token. |
| Assertion | A rule that must be true for the test to pass. | Actual status code must equal 201. |
| Payload / body | The JSON data sent or received. | {"email":"qa@example.com"} |
| Header | Metadata about the HTTP request/response. | Content-Type: application/json |
| Schema | Rules describing the allowed structure and types of JSON. | id is integer; email is string; token is required. |

# 1. Before writing the tests: project setup

## 1.1 Suggested Maven structure

```
rest-assured-learning/
 pom.xml
 src/
  test/
   java/
    tests/
     SignInApiTest.java
     EmployeeApiTest.java
   resources/
    payloads/
     signInRequestPayload.json
    schemas/
     signInResponseSchema.json
```

Why separate files? Java test code belongs in src/test/java. Reusable request JSON and response schemas belong in src/test/resources. This keeps test logic separate from test data and is easier to maintain.

## 1.2 Maven dependencies

Use compatible versions across REST Assured modules. The example below uses REST Assured 6.1.0 and TestNG 7.12.0 as a version snapshot checked for this guide. REST Assured 6.x requires Java 17 or newer; if your project runs on an older Java version, choose a compatible REST Assured 5.x version instead.

```
<dependencies>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>6.1.0</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>json-schema-validator</artifactId>
    <version>6.1.0</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.12.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

**Important** The schema-validator module version should match your REST Assured version. For projects that already have dependencies, avoid blindly adding duplicate versions; inspect pom.xml first.

## 1.3 What are the imports for?

```
import java.io.File;
import java.util.List;
import java.util.Map;

import org.hamcrest.MatcherAssert;
import org.hamcrest.Matchers;
import org.testng.annotations.Test;

import io.restassured.RestAssured;
import io.restassured.path.json.JsonPath;
import io.restassured.response.Response;
import io.restassured.specification.RequestSpecification;
import io.restassured.module.jsv.JsonSchemaValidator;
```

| Import | Why it appears |
|---|---|
| File | Represents a JSON payload or schema file on disk. |
| List | Represents a collection such as the JSON images array. |
| Map | Represents key/value data such as a JSON object. |
| MatcherAssert / Matchers | Performs Hamcrest assertions and supplies matching rules. |
| @Test | Tells TestNG that a method should run as a test. |
| RestAssured / RequestSpecification | Builds and sends HTTP requests. |
| Response | Stores status, headers, timing, and body returned by the API. |
| JsonPath | Reads values from a JSON response. |
| JsonSchemaValidator | Checks the body against a JSON Schema. |

# 2. Understanding assertions and Hamcrest

## 2.1 What is an assertion?

An assertion compares an actual result with an expected rule. If the rule is true, the assertion passes. If false, the test fails and TestNG reports the failure. Assertions are not just print statements: printing “200” does not prove the status is correct.

```
int actualStatus = 200;
MatcherAssert.assertThat(actualStatus, Matchers.equalTo(200));
```

Read this as: “Assert that the actualStatus is equal to 200.” If actualStatus is 500, the test fails.

## 2.2 Correct assertion syntax

```
MatcherAssert.assertThat(actualValue, Matchers.equalTo(expectedValue));
```

**Correction from the notes** Use MatcherAssert.assertThat(...) to perform an assertion. Matchers provides matchers such as equalTo(), lessThan(), and containsString(). Also use containsString(), not containString().

## 2.3 Common Hamcrest matchers

| Matcher | Meaning | Example | Typical use case |
|---|---|---|---|
| equalTo(x) | Exactly equals expected value. | equalTo(200) | Status code or role. |
| is(x) | Readable wrapper around a matcher/value. | is("ACTIVE") | Make simple assertion intent clearer. |
| greaterThan(x) | Must be greater than x. | greaterThan(0) | Price or item count. |
| greaterThanOrEqualTo(x) | Must be x or above. | greaterThanOrEqualTo(18) | Minimum eligible age. |
| lessThan(x) | Must be below x. | lessThan(5000L) | Response-time budget in ms. |
| lessThanOrEqualTo(x) | Must not exceed x. | lessThanOrEqualTo(100) | Discount cannot exceed 100%. |
| equalToIgnoringCase(x) | Text matches ignoring case. | equalToIgnoringCase("done") | Statuses with case-insensitive contract. |
| equalToIgnoringWhiteSpace(x) | Text matches after whitespace normalisation. | equalToIgnoringWhiteSpace("Order placed") | Human-readable text with variable spaces. |
| startsWith(x) | Text begins with x. | startsWith("Bearer ") | Authorization token prefix. |
| endsWith(x) | Text ends with x. | endsWith(".jpeg") | Image URL extension. |
| containsString(x) | Text includes substring x. | containsString("application/json") | Content-Type check. |
| hasItem(x) | Collection contains one matching item. | hasItem("redis") | One skill is present. |
| hasItems(a,b) | Collection contains each listed item. | hasItems("redis","java") | Required items are present. |
| notNullValue() | Value is not null. | notNullValue() | Required object ID exists. |
| instanceOf(Class) | Value is of expected Java class. | instanceOf(String.class) | Response field is a String. |
| hasKey(k) | Map contains key k. | hasKey("email") | Required key exists. |
| hasValue(v) | Map contains value v. | hasValue("ACTIVE") | Map includes a status value. |
| hasEntry(k,v) | Map contains a key/value pair. | hasEntry("status","ACTIVE") | Check a simple map entry. |

### 2.4 How to choose the matcher

```
// Numeric comparison: use a numeric type.
MatcherAssert.assertThat(price, Matchers.greaterThan(0.0));

// String comparison: use String.class, not "String".
MatcherAssert.assertThat(title, Matchers.instanceOf(String.class));

// Collection assertion.
MatcherAssert.assertThat(skills, Matchers.hasItem("java"));

// Map assertions.
MatcherAssert.assertThat(userMap, Matchers.hasKey("email"));
MatcherAssert.assertThat(userMap, Matchers.hasEntry("status", "ACTIVE"));
```

**Beginner tip** Match the Java type on both sides. A price may be Integer, Long, Float, or Double depending on the API and extraction method. Use the same compatible numeric type in the comparison, for example 0.0 for a double.

# 3. Validate status code, status line, time and headers

### 3.1 Status code

The status code is the numeric HTTP result. Common examples include 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, and 500 Internal Server Error. Choose the expected code from the API contract, not from habit.

```
int code = response.statusCode();
MatcherAssert.assertThat(code, Matchers.equalTo(200));
```

Real scenario: when retrieving an existing product, the API contract says it should return 200. If the API returns 404, the assertion fails and reveals a functional problem or incorrect test data.

### 3.2 Status line

```
String line = response.statusLine();
MatcherAssert.assertThat(line, Matchers.containsString("OK"));
```

The status line combines the protocol, status code, and phrase, although exact output can vary. In most automated tests the numeric status code is the stronger and less brittle assertion. Use status-line assertions only when your team has a real requirement to validate it.

### 3.3 Response time

```
long timeMs = response.time();
MatcherAssert.assertThat(timeMs, Matchers.lessThan(5000L));
```

This example says the response should take less than 5,000 milliseconds (5 seconds). A timing assertion is a coarse test threshold; it is not a full performance or load test. Run it in a stable test environment and choose a limit based on the service-level requirement.

### 3.4 Response headers

```
String contentType = response.getHeader("Content-Type");
MatcherAssert.assertThat(contentType,
      Matchers.containsString("application/json"));
```

Content-Type describes the format of the returned body. For JSON APIs it commonly includes application/json, sometimes with a charset parameter.

```
String cacheControl = response.getHeader("Cache-Control");
// Only assert a specific cache policy when the API contract defines it.
MatcherAssert.assertThat(cacheControl, Matchers.notNullValue());
```

**Do not hard-code every server header** Headers such as Server, Date, or infrastructure-specific names may change when a load balancer, CDN, proxy, or hosting platform changes. Prefer stable contract headers such as Content-Type, Location for created resources, or documented rate-limit headers.

### 3.5 A compact response-metadata test

```
@Test
public void verifyProductResponseMetadata() {
    Response response = RestAssured.given()
            .get("https://api.example.test/products/28");

    MatcherAssert.assertThat(response.statusCode(),
            Matchers.equalTo(200));
    MatcherAssert.assertThat(response.getHeader("Content-Type"),
            Matchers.containsString("application/json"));
    MatcherAssert.assertThat(response.time(),
            Matchers.lessThan(5000L));
}
```

The example domain api.example.test is a placeholder. Replace it with an API URL available in your test environment.

## 4. POST sign-in API: request body from a JSON file

### 4.1 Real-world scenario

A web application has a sign-in endpoint. A user submits email and password. The QA test sends the same kind of JSON request, then checks that the API returned the documented success code, JSON content type, expected identity fields, and a non-empty token. Never print or share real credentials or access tokens in logs.

### 4.2 Example request payload

**src/test/resources/payloads/signInRequestPayload.json**

```
{
  "email": "qa.user@example.com",
  "password": "TrainingPassword123!"
}
```

This is sample training data, not a real account. In a real framework, store test credentials securely or use environment variables/secret management rather than committing secrets to Git.

### 4.3 Step-by-step flow

| Step | Code / action | Why it matters |
|---|---|---|
| 1 | Create RequestSpecification. | Start building the HTTP request. |
| 2 | Set Content-Type to application/json. | Tell the server the request body is JSON. |
| 3 | Read JSON payload file. | Keep test data separate from test code. |
| 4 | Attach payload using body(...). | Send the JSON to the API. |
| 5 | Call post(endpoint). | Execute the POST request. |
| 6 | Assert status and headers. | Check HTTP-level expectations. |
| 7 | Read response JsonPath. | Extract id, email, token, and other values. |
| 8 | Validate schema. | Check the overall response contract. |

## 4.4 Complete beginner-friendly TestNG example

```
package tests;

import java.io.File;
import org.hamcrest.MatcherAssert;
import org.hamcrest.Matchers;
import org.testng.annotations.Test;
import io.restassured.RestAssured;
import io.restassured.path.json.JsonPath;
import io.restassured.response.Response;
import io.restassured.specification.RequestSpecification;

public class SignInApiTest {

  @Test
  public void signInWithValidCredentials() {
    // 1. Prepare the request.
    RequestSpecification request = RestAssured.given();
    request.contentType("application/json");

    // 2. Load the request JSON file.
    File payload = new File(
      "src/test/resources/payloads/signInRequestPayload.json");
    request.body(payload);

    // 3. Send the request to the application's API.
    Response response = request.post(
      "http://localhost:9197/auth/signin");

    // 4. Validate the HTTP response.
    MatcherAssert.assertThat(response.statusCode(),
      Matchers.equalTo(201)); // Use only if contract expects 201.
    MatcherAssert.assertThat(response.getHeader("Content-Type"),
      Matchers.containsString("application/json"));
    MatcherAssert.assertThat(response.time(),
      Matchers.lessThan(5000L));

    // 5. Extract fields from the JSON response.
    JsonPath json = response.jsonPath();
    int id = json.getInt("id");
    String email = json.getString("email");
    String token = json.getString("token");

    // 6. Validate important business fields.
    MatcherAssert.assertThat(id, Matchers.greaterThan(0));
    MatcherAssert.assertThat(email,
      Matchers.equalTo("qa.user@example.com"));
    MatcherAssert.assertThat(token,
      Matchers.not(Matchers.isEmptyOrNullString()));
  }
}
```

## 4.5 Line-by-line explanation

- @Test: TestNG discovers and executes this method as a test.
- RestAssured.given(): creates a request builder. It is where headers, query parameters, body, cookies, and authentication can be configured.
- contentType("application/json"): sets the outgoing request Content-Type header. Do not confuse it with Accept; Accept says what response media type the client prefers.
- new File(...): points to the JSON data file relative to the project working directory. The path must exist when the test runs.
- request.body(payload): attaches the JSON file contents as the request body.
- request.post(url): sends the POST request and returns a Response object.

- response.statusCode(): reads the returned HTTP status as an integer.
- response.getHeader(...): reads a specific response header. Header names are case-insensitive in HTTP.
- response.time(): returns an elapsed time measurement in milliseconds by default in this usage.
- response.jsonPath(): creates a helper to extract fields from the JSON body. json.getString("token") reads the root-level token property.

**Status code warning** The class notes use 201. A sign-in API often returns 200, but neither is universally correct. Assert the exact code documented by your API. For invalid credentials, assert the error status and error response contract instead.

# 5. JSON Schema validation

## 5.1 What is a JSON Schema?

A JSON Schema is a contract for the shape of JSON data. It can define the root type, permitted properties, required keys, data types, patterns, and other constraints. Schema validation does not replace business assertions; use both.

Real scenario: a sign-in endpoint may return the expected status but accidentally omit token or return id as a string instead of an integer. A schema test can catch that contract regression.

## 5.2 Example sign-in response

```
{
 "id": 42,
 "email": "qa.user@example.com",
 "firstName": "Kiran",
 "lastName": "Rao",
 "role": "USER",
 "token": "sample-token-not-for-production"
}
```

## 5.3 Corrected schema

**src/test/resources/schemas/signInResponseSchema.json**

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "type": "object",
 "additionalProperties": false,
 "properties": {
  "id": { "type": "integer", "minimum": 1 },
  "email": { "type": "string", "format": "email" },
  "firstName": {
   "type": "string",
   "pattern": "^[A-Za-z]+$"
  },
  "lastName": { "type": "string" },
  "role": { "type": "string" },
  "token": { "type": "string", "minLength": 1 }
 },
 "required": [
  "id", "email", "firstName", "lastName", "role", "token"
 ]
}
```

## 5.4 Explanation of the rules

| Rule | Meaning | Why the API team might care |
|---|---|---|
| "type": "object" | The response root must be a JSON object. | A list or plain string response is an API contract break. |
| "properties" | Lists fields and validation rules. | Documents the expected response structure. |
| "required" | These keys must exist. | A missing token might prevent the client |

| | | |
|---|---|---|
| | | from continuing. |
| "additionalProperties": false | Rejects unspecified fields. | Useful for strict contracts; can be too strict if API intentionally adds fields. |
| "type": "integer" | Value must be an integer. | User ID should not silently become a string. |
| "minimum": 1 | Integer must be at least 1. | Catches invalid/non-positive IDs if the contract requires positive IDs. |
| "pattern" | String must match a regular expression. | Restricts firstName to letters in this example. |
| "minLength": 1 | String must contain at least one character. | Prevents blank token values. |

**Important correction to the source schema** The source places pattern on an integer id. JSON Schema pattern applies to strings, not integers. Use minimum for a positive numeric ID, or declare the ID as a string if the API actually returns it as text.

### 5.5 Validate the response against the schema

```
import java.io.File;
import io.restassured.module.jsv.JsonSchemaValidator;

String responseBody = response.body().asString();
File schemaFile = new File(
   "src/test/resources/schemas/signInResponseSchema.json");

MatcherAssert.assertThat(responseBody,
   JsonSchemaValidator.matchesJsonSchema(schemaFile));
```

The validator compares the returned body with the schema rules. If a required field is missing, a data type changes, or an unexpected field appears while additionalProperties is false, the assertion fails.

### 5.6 Schema validation vs business assertions

| Schema validation asks… | Business assertion asks… |
|---|---|
| Does id exist and have integer type? | Is this the correct user ID for the account used? |
| Is token a non-empty string? | Does this sign-in flow provide a token usable by the next request? |
| Is email represented as a string? | Does the response email match the submitted email? |
| Are required fields present? | Is role equal to USER for this test account? |

# 6. JsonPath: extracting values from JSON

### 6.1 The response is JSON; JsonPath reads values

Suppose the response body is the following JSON. Property names such as id, email, token, category, and images are keys. The data after each colon is its value.

```
{
 "id": 28,
 "title": "Classic T-shirt",
 "price": 25.5,
 "available": true,
 "category": {
  "id": 3,
  "name": "Clothes"
 },
 "images": [
  "https://cdn.example.test/item-1.jpeg",
  "https://cdn.example.test/item-2.jpeg"
 ]
}
```

## 6.2 Create a JsonPath object

```
JsonPath json = response.jsonPath();
int id = json.getInt("id");
String title = json.getString("title");
String categoryName = json.getString("category.name");
String firstImage = json.getString("images[0]");
```

| Expression | What it reads | Expected Java type |
|---|---|---|
| getInt("id") | Root property id. | int |
| getString("title") | Root property title. | String |
| getDouble("price") | Decimal price. | double |
| getBoolean("available") | Boolean flag. | boolean |
| getInt("category.id") | id inside category object. | int |
| getString("category.name") | name inside category object. | String |
| getString("images[0]") | First array element; indexes start at zero. | String |
| getList("images") | All values from images array. | List<?> or List<String> |
| getMap("category") | Nested category object as a map. | Map<String, ?> |
| get("field") | Value when type is not yet convenient to specify. | Object / inferred value |

## 6.3 Common JsonPath getters

Your class notes list getByte(), getShort(), getInt(), getLong(), getBoolean(), getString(), getList(), getMap(), get(), getFloat(), and getDouble(). Use the getter that matches the data contract. You do not usually need getChar() for JSON because JSON has strings rather than a dedicated character type.

```
JsonPath json = response.jsonPath();
int userId = json.getInt("id");
long mobileNumber = json.getLong("mobileNumber");
boolean active = json.getBoolean("active");
String username = json.getString("username");
List<String> skills = json.getList("skills");
Map<String, Object> address = json.getMap("address");
```

**Do not assert notNullValue() on a primitive after extraction** If you call getInt("id"), Java gives you a primitive int, which cannot be null. To test presence/nullability directly, assert on an Object/String value or verify that the JSON path exists and then validate the expected numeric value.

### 6.4 Extracting arrays and nested arrays

```
// Body: { "skills": ["java", "sql", "api-testing"] }
List<String> skills = json.getList("skills");
MatcherAssert.assertThat(skills, Matchers.hasItem("api-testing"));
MatcherAssert.assertThat(skills,
    Matchers.hasItems("java", "sql"));

// If the body has: { "images": [{"type":"image/jpeg"}] }
String type = json.getString("images[0].type");
MatcherAssert.assertThat(type, Matchers.equalTo("image/jpeg"));
```

Array notation [0] refers to the first element. Dot notation such as category.name moves down into a nested object. A wrong path often returns null or causes a type-conversion failure; inspect the actual response body when a path fails.

## 7. Full annotated example: employee API GET test

### 7.1 Scenario

A company's employee-management application exposes GET /api/employees/3. A QA engineer wants to confirm the service returns an existing employee with valid metadata and a usable response body. The URL is from the class notes and assumes a local application is running on localhost:9192.

### 7.2 Cleaner TestNG implementation

```
package tests;

import org.hamcrest.MatcherAssert;
import org.hamcrest.Matchers;
import org.testng.annotations.Test;
import io.restassured.RestAssured;
import io.restassured.path.json.JsonPath;
import io.restassured.response.Response;

public class EmployeeApiTest {

    @Test
    public void getEmployeeById() {
        Response response = RestAssured.given()
            .accept("application/json")
            .get("http://localhost:9192/api/employees/3");

        // HTTP-level checks
        MatcherAssert.assertThat(response.statusCode(),
            Matchers.equalTo(200));
        MatcherAssert.assertThat(response.getHeader("Content-Type"),
            Matchers.containsString("application/json"));
        MatcherAssert.assertThat(response.time(),
            Matchers.lessThan(5000L));

        // Body-level checks: adjust field names to the real contract.
        JsonPath json = response.jsonPath();
        MatcherAssert.assertThat(json.get("username"),
            Matchers.instanceOf(String.class));
        MatcherAssert.assertThat(json.get("country"),
            Matchers.notNullValue());
        MatcherAssert.assertThat(json.get("mobileNumber"),
            Matchers.notNullValue());
    }
}
```

### 7.3 Understand the response checks

- accept("application/json") sets the Accept request header: the test prefers a JSON response.

- get(url) sends an HTTP GET request; this request does not include a JSON body in the example.
- statusCode() checks the HTTP outcome before the body assertions are interpreted.
- get("username") extracts the field as a generic Object. instanceOf(String.class) validates the Java representation.
- notNullValue() is used here on generic Object values so the test can check that those fields are present/non-null. A stricter test should also check formats, ranges, and business expectations.

### 7.4 Common bug in the original sample

The source extracts country, mobileNumber, and username. Do not assume the mobile number is a long without checking the API contract: phone numbers are often best represented as strings because they may contain leading zeros or a +country code. If the API defines it as a number, use getLong(); otherwise validate it as a string.

## 8. Real-world scenario: e-commerce product details

### 8.1 Business requirement

When a customer opens a product detail page, the API must return the correct product, a positive price, a category, and a list of image URLs. A successful HTTP response with incorrect product data is still a defect.

### 8.2 Example response

```
{
 "id": 28,
 "title": "Classic T-shirt",
 "price": 25.5,
 "description": "Cotton crew-neck T-shirt",
 "category": { "id": 3, "name": "Clothes" },
 "images": [
  "https://cdn.example.test/classic-1.jpeg",
  "https://cdn.example.test/classic-2.jpeg"
 ]
}
```

### 8.3 Test code

```
@Test
public void productDetailsShouldBeValid() {
   Response response = RestAssured.given()
      .accept("application/json")
      .get("https://api.escuelajs.co/api/v1/products/28");

   MatcherAssert.assertThat(response.statusCode(),
      Matchers.equalTo(200));

   JsonPath json = response.jsonPath();
   MatcherAssert.assertThat(json.getInt("id"),
      Matchers.equalTo(28));
   MatcherAssert.assertThat(json.getString("title"),
      Matchers.not(Matchers.isEmptyOrNullString()));
   MatcherAssert.assertThat(json.getDouble("price"),
      Matchers.greaterThan(0.0));
   MatcherAssert.assertThat(json.getString("category.name"),
      Matchers.not(Matchers.isEmptyOrNullString()));

   java.util.List<String> images = json.getList("images");
   MatcherAssert.assertThat(images, Matchers.not(Matchers.empty()));
   MatcherAssert.assertThat(images,
      Matchers.hasItem(Matchers.containsString(".jpeg")));
}
```

**Use stable assertions** This code demonstrates how to test field rules, but live public test APIs can change, throttle requests, or be temporarily unavailable. In an employer project, prefer the company's approved QA environment and test data. Do not make your build dependent on an external demo API being healthy.

### 8.4 Typical bugs this test could catch

| Defect | What the assertion catches |
|---|---|
| API returns product 27 when 28 was requested. | id assertion fails. |
| Price accidentally becomes zero or negative. | greaterThan(0.0) fails. |
| Category name is omitted. | not(empty/null string) fails. |
| Images are an empty array. | not(empty()) fails. |
| Image URLs are malformed or wrong format. | URL/extension assertion can fail; production tests may validate the full URI. |

# 9. Real-world scenario: rate limits, headers and performance

### 9.1 Scenario

A public API limits the number of calls a client can make in a window. The API contract might document a rate-limit header, and the team needs to ensure the header is present and follows the expected format.

```
String remaining = response.getHeader("X-RateLimit-Remaining");
MatcherAssert.assertThat(remaining, Matchers.notNullValue());

// If the API documents this header as an integer:
int remainingCount = Integer.parseInt(remaining);
MatcherAssert.assertThat(remainingCount, Matchers.greaterThanOrEqualTo(0));
```

The exact header name varies across APIs. Do not copy a header name from an unrelated service. Also, a rate-limit header may count down between calls, so asserting it equals a fixed number can create a flaky test.

### 9.2 Response-time checks

```
long elapsedMs = response.time();
MatcherAssert.assertThat("API should respond in under 5 seconds",
      elapsedMs, Matchers.lessThan(5000L));
```

The string argument makes the assertion failure easier to understand. If the threshold is missed, inspect environment load, network latency, API logs, and whether the test needs a less flaky threshold. A single request-time assertion does not replace performance testing with load and percentile analysis.

### 9.3 Example: create-resource Location header

Some APIs return 201 Created and a Location header pointing to the new resource. Only assert this if the API contract promises it.

```
MatcherAssert.assertThat(response.statusCode(), Matchers.equalTo(201));
String location = response.getHeader("Location");
MatcherAssert.assertThat(location, Matchers.not(Matchers.isEmptyOrNullString()));
```

# 10. Negative testing: the most valuable extra checks

Positive testing asks "Does a valid request work?" Negative testing asks "Does the API reject bad input safely and consistently?" The exact response code and error shape must come from the API contract.

| Scenario | Example request/input | Expected check (contract-dependent) |
|---|---|---|
| Wrong password | Correct email, wrong password. | 401 or documented auth error; no token is issued. |
| Missing password | Email only. | 400/422; field-validation error is clear. |

| | | |
|---|---|---|
| Malformed JSON | Truncated/invalid JSON text. | 4xx response; server should not crash. |
| Unknown product ID | GET /products/99999999. | 404 or documented not-found response. |
| Invalid ID format | GET /products/abc. | 400/404 depending on route/contract. |
| Missing auth token | Call protected endpoint without Authorization. | 401/403; sensitive data is not returned. |
| Expired token | Use an intentionally expired test token. | 401/403 and a documented error response. |

### 10.1 Example: invalid sign-in

```
@Test
public void signInShouldRejectWrongPassword() {
   String body = "{\"email\":\"qa.user@example.com\","
           + "\"password\":\"WrongPassword\"}";

   Response response = RestAssured.given()
      .contentType("application/json")
      .body(body)
      .post("http://localhost:9197/auth/signin");

   // Example only: confirm the correct code with your API contract.
   MatcherAssert.assertThat(response.statusCode(),
      Matchers.equalTo(401));

   JsonPath json = response.jsonPath();
   MatcherAssert.assertThat(json.getString("token"),
      Matchers.nullValue());
}
```

This example assumes the error response has a token field set to null. Many APIs omit the token field altogether or use a different error schema. Adjust the assertion to the actual documented contract; do not force your API to match this invented example.

### 10.2 Common security assertions

- Invalid login does not return an access token.
- Error messages do not reveal whether a password alone is correct unless product/security requirements explicitly permit that distinction.
- Protected endpoints reject missing or invalid credentials.
- Logs and test reports do not print real passwords, bearer tokens, or other secrets.

## 11. Common mistakes in the source code and how to fix them

| Mistake | Why it is a problem | Better version |
|---|---|---|
| Matchers.assertThat(...) | assertThat belongs to MatcherAssert. | MatcherAssert.assertThat(actual, Matchers.equalTo(expected)); |
| Matchers.containString(...) | Method name is misspelled. | Matchers.containsString("application/json") |
| greaterThanAndEqualTo(...) | This is not the Hamcrest method name. | Matchers.greaterThanOrEqualTo(expected) |
| Matchers.instanceOf("String") | instanceOf expects a Class object, not text. | Matchers.instanceOf(String.class) |
| int id then notNullValue() | Primitive int cannot be null. | Assert id > 0; or assert raw Object is not null. |
| int price = json.getInt("id") | Reads the ID, not the price. | double price = json.getDouble("price") |
| pattern on integer in JSON Schema | pattern is a string constraint. | Use minimum/maximum for numbers. |
| rate-limit header compared to integer 100 | HTTP header values are strings and may change. | Assert non-null/parse integer/validate contract semantics. |

| | | |
|---|---|---|
| Server header must equal Heroku | Infrastructure can change the header. | Assert only stable, required contract headers. |
| Hard-coded Windows file path | Other machines do not have that directory. | Use src/test/resources or classpath resource loading. |
| Only printing response body | Print output alone does not make a test fail. | Write assertions for required behaviour. |
| Assuming all successful POSTs return 201 | Status code depends on endpoint contract. | Check the API specification. |
| Strict additionalProperties:false everywhere | A harmless new field might break tests. | Use strict schemas only where strict contract enforcement is intended. |

### A note on generic objects and null checks

```
// This is valid when value is an Object:
Object token = json.get("token");
MatcherAssert.assertThat(token, Matchers.notNullValue());

// This checks a string for non-empty value:
String email = json.getString("email");
MatcherAssert.assertThat(email,
   Matchers.not(Matchers.isEmptyOrNullString()));

// This checks a numeric business rule:
int id = json.getInt("id");
MatcherAssert.assertThat(id, Matchers.greaterThan(0));
```

# 12. Build a reusable response-validation pattern

## 12.1 Why reusable patterns help

When every test repeats many lines for status, Content-Type, and timing, maintenance becomes difficult. REST Assured supports fluent assertions; helpers can also centralize checks. Do not hide every assertion inside one giant helper, because the test should still clearly show what it verifies.

## 12.2 Fluent REST Assured style

```
RestAssured.given()
   .accept("application/json")
.when()
   .get("http://localhost:9192/api/employees/3")
.then()
   .statusCode(200)
   .contentType("application/json")
   .time(Matchers.lessThan(5000L));
```

This style combines request execution and assertions. Use it when you only need to validate the response directly. Use a Response variable and JsonPath when you need to store/extract values, chain calls, or debug step by step.

## 12.3 Example helper for metadata

```
public final class ResponseChecks {
   private ResponseChecks() { }

   public static void assertJsonSuccess(Response response, int expectedCode) {
      MatcherAssert.assertThat(response.statusCode(),
         Matchers.equalTo(expectedCode));
      MatcherAssert.assertThat(response.getHeader("Content-Type"),
         Matchers.containsString("application/json"));
   }
}
```

Keep helpers small, predictable, and documented. Avoid using this helper for endpoints that intentionally return non-JSON, or success responses with a different content type.

# 13. Hands-on exercises

Complete these before reading the answer key. For every exercise, run at least one passing case and deliberately change an expected value to confirm the test fails.

### Exercise 1 - status and content type

Send a GET request to a working product endpoint. Assert status 200 and Content-Type contains application/json. Print the body only to help debugging; do not use printing as the test assertion.

### Exercise 2 - numeric rules

Extract price from a product JSON response as a double and assert it is greater than 0 and less than or equal to 10,000.

### Exercise 3 - strings

Extract a product title. Assert it is not null/empty and starts with the first word expected by your test data. Make the expected title appropriate for the environment you run against.

### Exercise 4 - nested JSON

Given category: {"id":3,"name":"Clothes"}, extract category.name and category.id. Assert the name is “Clothes” and the ID is greater than zero.

### Exercise 5 - arrays

Extract skills from {"skills":["java","sql","api-testing"]}. Assert that java and api-testing are present.

### Exercise 6 - map keys

Read the response as a Map and assert it contains keys id, email, and token. Note: getMap("") may give a raw map; consider Map<String,Object> or an Object depending on type safety.

### Exercise 7 - schema failure

Create a schema requiring id and email. Run it against a body missing email. Confirm the schema assertion fails for the intended reason.

### Exercise 8 - POST request from file

Create signInRequestPayload.json in src/test/resources/payloads. Send it as a POST body with Content-Type application/json. Validate the expected status and content type.

### Exercise 9 - invalid credentials

Use a dedicated invalid-password test account/data. Assert the documented error status, verify no access token is returned, and ensure the error shape matches the contract.

### Exercise 10 - response time

Assert a non-critical read endpoint responds under a documented threshold. Run it multiple times. If it fails intermittently, investigate the environment before widening the threshold.

### Exercise 11 - find and fix the bug

```
double price = json.getDouble("id");
MatcherAssert.assertThat(price, Matchers.greaterThan(0.0));
```

Question: What is wrong, and what should the extraction path be?

### Exercise 12 - build a complete mini test

Create a TestNG test for an employee GET endpoint. Check status, content type, response time, username type, country non-null, and mobile number according to the API contract. Keep the API URL configurable if possible.

## 14. Worked solutions

### Solution 1 - status and content type

```
Response response = RestAssured.given()
    .accept("application/json")
    .get(baseUrl + "/products/28");

MatcherAssert.assertThat(response.statusCode(), Matchers.equalTo(200));
MatcherAssert.assertThat(response.getHeader("Content-Type"),
    Matchers.containsString("application/json"));
```

### Solution 2 - numeric rules

```
double price = response.jsonPath().getDouble("price");
MatcherAssert.assertThat(price, Matchers.greaterThan(0.0));
MatcherAssert.assertThat(price, Matchers.lessThanOrEqualTo(10000.0));
```

### Solution 3 - strings

```
String title = response.jsonPath().getString("title");
MatcherAssert.assertThat(title,
    Matchers.not(Matchers.isEmptyOrNullString()));
MatcherAssert.assertThat(title, Matchers.startsWith("Classic"));
```

The word Classic is only an example. Use an expected value that matches the controlled test data you request.

### Solution 4 - nested JSON

```
JsonPath json = response.jsonPath();
MatcherAssert.assertThat(json.getString("category.name"),
    Matchers.equalTo("Clothes"));
MatcherAssert.assertThat(json.getInt("category.id"),
    Matchers.greaterThan(0));
```

### Solution 5 - arrays

```
List<String> skills = json.getList("skills");
MatcherAssert.assertThat(skills, Matchers.hasItem("java"));
MatcherAssert.assertThat(skills, Matchers.hasItem("api-testing"));
```

### Solution 6 - map keys

```
Map<String, Object> body = response.jsonPath().getMap("");
MatcherAssert.assertThat(body, Matchers.hasKey("id"));
MatcherAssert.assertThat(body, Matchers.hasKey("email"));
MatcherAssert.assertThat(body, Matchers.hasKey("token"));
```

### Solution 7 - schema

```
String body = "{\"id\": 5}"; // email intentionally missing
MatcherAssert.assertThat(body,
    JsonSchemaValidator.matchesJsonSchema(schemaFile));
// Expected outcome: this assertion fails because email is required.
```

### Solution 11 - fix the bug

```
// Wrong: this extracts the ID and stores it in price.
double price = json.getDouble("id");

// Correct:
double price = json.getDouble("price");
```

### Solution strategy for exercises 8, 9, 10 and 12

- Use the endpoint and expected status defined by your API contract; do not assume the sample localhost service is running.
- For negative tests, assert the actual documented error contract and check that sensitive success data is absent.
- For response time, set a reasonable environment-specific budget and keep performance/load testing separate.
- For employee fields, first inspect the body and API contract to decide whether mobileNumber is numeric or a string.

## 15. Debugging checklist

| Symptom | Check next |
|---|---|
| Connection refused | Is the local API/service running at the right port? Is the URL correct? |
| 404 Not Found | Does the endpoint path and resource ID exist? Is a context path missing? |
| 401 / 403 | Is authentication required? Is the token valid? Does the role have permission? |
| 415 Unsupported Media Type | Did you set Content-Type correctly for the request body? |
| 400 Bad Request | Is the JSON valid? Are required fields and data types correct? |
| JsonPath gives null | Print/inspect the response body; confirm the field name and nesting. |
| ClassCastException / conversion error | Does the field really have the type your getter expects? |
| Schema assertion fails | Which required field, type, pattern, or extra property is inconsistent? |
| Intermittent response-time failure | Review environment stability, test threshold, logs, and whether the test is overloaded. |
| File not found | Check the file path relative to the Maven project root or use classpath resource loading. |

### A safe debugging pattern

```
System.out.println("Status: " + response.statusCode());
System.out.println("Content-Type: " + response.getHeader("Content-Type"));
System.out.println(response.body().asPrettyString());
```

**Protect secrets** Do not print credentials, Authorization headers, cookies, or access tokens in CI logs. For sensitive APIs, use redacted logging or log only safe fields such as status code and request correlation ID.

# 16. Best practices to build professional habits

**Assert behaviour, not just availability:** A 200 response can contain the wrong user or product. Validate the business fields that matter.

**Keep expected values meaningful:** For a product ID of 28, assert id == 28 rather than only asserting id > 0.

**Use explicit, readable tests:** A future teammate should understand test intent from the method name and assertions.

**Separate test data:** Keep payloads and schemas under src/test/resources; avoid absolute workstation paths.

**Prefer stable contracts:** Do not over-assert server implementation headers that can change without breaking the API contract.

**Use schema and business assertions together:** Schema protects structure; business assertions verify meaning.

**Avoid public API fragility:** Use controlled environments and deterministic fixtures for CI.

**Avoid secrets in source control:** Use safe test credentials and environment/secret configuration.

**Keep one test focused:** Do not combine too many unrelated assertions into one test if a single failure would make diagnosis unclear.

**Check failure messages:** Add useful assertion messages when it makes failures faster to diagnose.

## Quick revision sheet

| Need to… | Use… |
|---|---|
| Send request | RestAssured.given() then get/post/put/patch/delete |
| Store response | Response response = ... |
| Read code / line / time | statusCode() / statusLine() / time() |
| Read header | getHeader("Content-Type") |
| Read response JSON | response.jsonPath() |
| Read nested property | json.getString("category.name") |
| Read array element | json.getString("images[0]") |
| Read complete array | json.getList("images") |
| Assert actual vs expected | MatcherAssert.assertThat(actual, Matchers.equalTo(expected)) |
| Validate schema | JsonSchemaValidator.matchesJsonSchema(schemaFile) |

# 17. Mini capstone: verify sign-in like an API QA engineer

Goal: implement a single TestNG method that sends the sign-in payload, verifies status and headers, validates the response schema, and checks important returned values. Use the sample localhost endpoint from the class only if that service runs on your machine.

## Acceptance criteria

- Request uses Content-Type application/json.
- Payload is read from src/test/resources/payloads/signInRequestPayload.json.
- Status code equals the API contract's expected value.
- Content-Type is JSON and response timing is under the chosen threshold.
- Response conforms to sign-in response schema.
- id is a positive integer; email equals the intended test account; token is non-empty.
- No real secret value is logged or committed.

## Extension tasks

- Add a test for incorrect password and validate the error response.
- Add a test for missing email and check the field-validation contract.

- Externalize baseUrl so localhost, QA, and staging can use the same test.
- Add a clear assertion message for each important business rule.
- Run the tests in Maven and inspect the Surefire/TestNG report.

**Your next milestone** Once this is comfortable, the next logical topics are request/response specifications, reusable base URI configuration, authentication token chaining, TestNG data providers, POJOs, logging, and API CRUD workflows.

# 18. Glossary

| Term | Meaning |
|---|---|
| API | A contract/interface that allows one software component to interact with another. |
| REST Assured | Java library used to write automated tests for HTTP APIs. |
| HTTP | Protocol used by clients and servers to exchange requests and responses. |
| Status code | Numeric result of a request, e.g. 200 or 404. |
| Header | Metadata sent with the HTTP request or response. |
| Payload/body | The actual data being sent or returned, often JSON. |
| Assertion | Executable check comparing actual behaviour with an expected rule. |
| Matcher | Reusable condition such as equalTo, hasItem, or containsString. |
| JsonPath | Expression/helper for locating data inside a JSON response. |
| JSON Schema | Rules defining accepted JSON structure, fields, types, and constraints. |
| TestNG | Java test framework that discovers, runs, and reports tests. |
| Negative test | A test that supplies invalid or unauthorized input and checks safe rejection. |
| Contract test | Test verifying that an API follows its documented request/response structure. |
| Flaky test | A test that passes/fails unpredictably without a meaningful product change. |

## Version references checked for this guide

REST Assured 6.1.0 release and Maven artifact listing: https://rest-assured.io/ and https://central.sonatype.com/artifact/io.rest-assured/rest-assured

TestNG Maven artifact listing: https://central.sonatype.com/artifact/org.testng/testng

Always confirm Java compatibility and dependency versions against your team's existing project before upgrading.